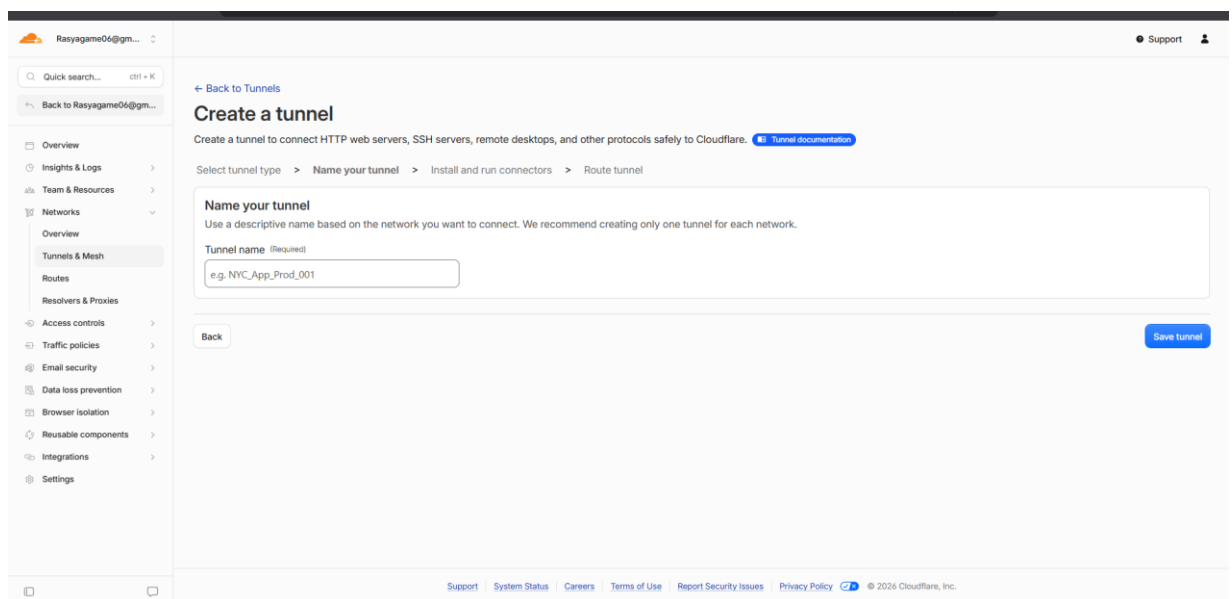


Konfigurasi dan Implementasi Cloudflare Tunnel (Zero Trust) via Docker

BAGIAN 1: Pembuatan Tunnel pada Dasbor Cloudflare

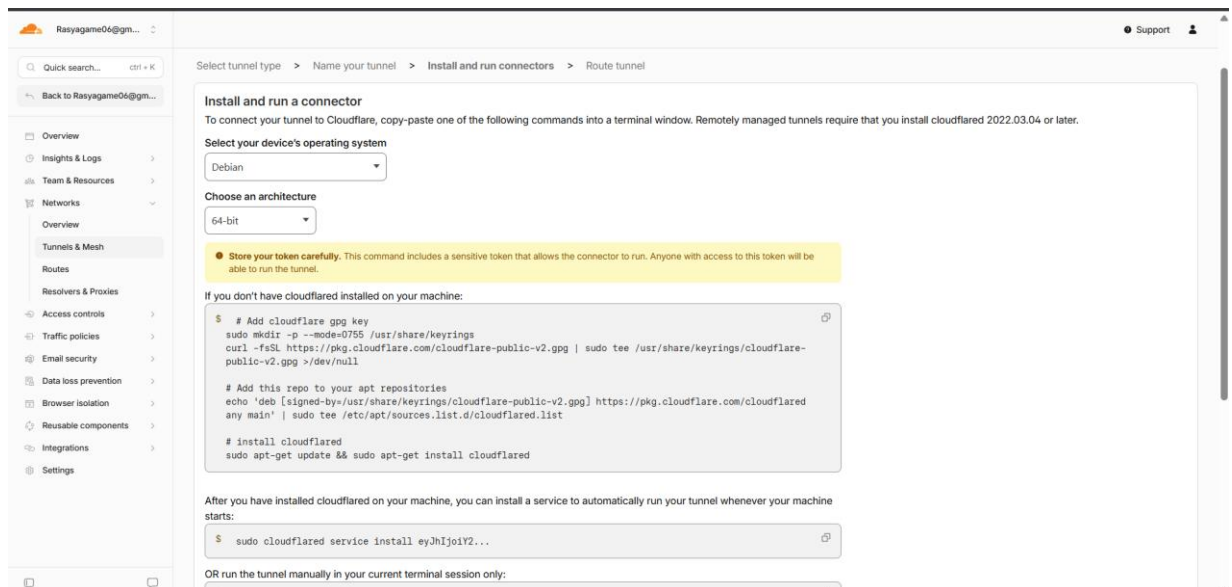
1. Akses Dasbor Zero Trust: Masuk ke dasbor utama Cloudflare dan navigasikan ke panel Zero Trust.
2. Inisiasi Tunnel Baru: Pada menu navigasi sebelah kiri, pilih Networks, kemudian klik Tunnels.
3. Pilih Tipe Konektor: Klik tombol Create a tunnel. Pada opsi pemilihan konektor, pilih Cloudflared lalu klik Next.
4. Pemberian Identitas: Masukkan nama yang representatif untuk tunnel tersebut (misalnya: ubuntu-server), kemudian klik Save tunnel.

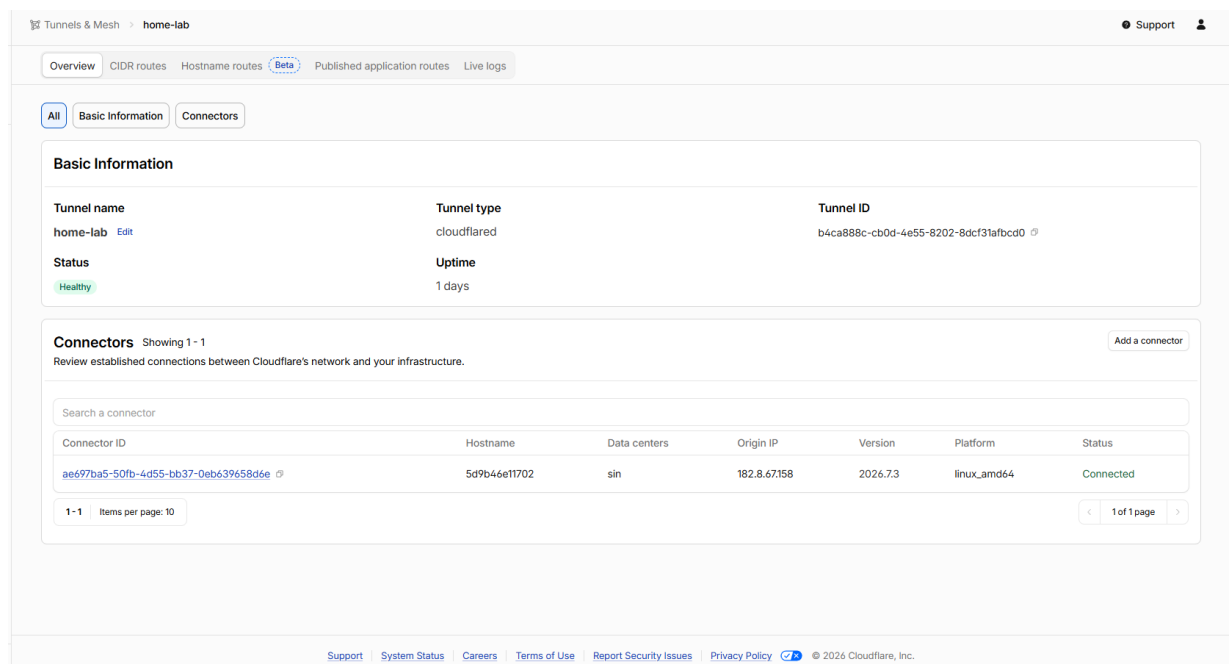
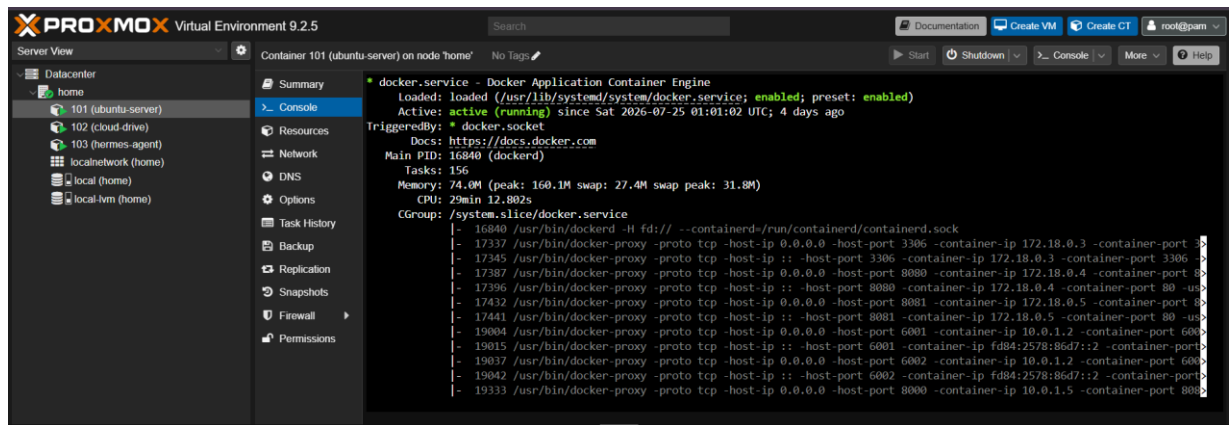


BAGIAN 2: Instalasi dan Konfigurasi Cloudflared pada Server Alpine Linux

1. Pengambilan Perintah Instalasi: Pada halaman "Install and run a connector", pilih environment Docker untuk mendapatkan perintah instalasi yang memuat token autentikasi unik. Salin perintah tersebut.
2. Akses Server Lokal: Buka terminal pada Virtual Machine (VM) Alpine Linux.
3. Eksekusi Docker Container: Jalankan perintah instalasi melalui Docker. Tambahkan parameter -d agar layanan berjalan di background dan --network host agar container dapat membaca port layanan lokal server. Format perintah yang dieksekusi adalah sebagai berikut:

```
docker run -d --network host cloudflare/cloudflared:latest tunnel --no-autoupdate run --token <TOKEN_RAHASIA_CLOUDFLARE>
```
4. Verifikasi Layanan: Lakukan pengecekan dengan perintah "docker ps -a" untuk memastikan container Cloudflared telah berjalan (Up).
5. Konfirmasi Koneksi: Kembali ke dasbor Cloudflare. Pastikan indikator konektor telah berubah menjadi Connected (berwarna hijau) yang menandakan server lokal telah terhubung ke jaringan Cloudflare, lalu klik Next.





BAGIAN 3: Konfigurasi Routing Public Hostname

1. Pengaturan Domain: Pada halaman Route traffic, konfigurasi bagian Public Hostnames untuk menentukan alamat akses publik web statis:
 - Subdomain: Isi dengan nama subdomain yang diinginkan (contoh: doc).
 - Domain: Pilih domain utama yang telah dikelola di Cloudflare (contoh: dkiuin.my.id).
2. Pengarahan Trafik (Service): Arahkan lalu lintas internet ke layanan web Nginx lokal dengan pengaturan berikut:
 - Type: Pilih HTTP.

- URL: Masukkan localhost:8081 (merujuk pada port Nginx yang berjalan di VM Alpine).

3. Penyimpanan: Klik Save tunnel untuk mengaktifkan seluruh konfigurasi.

4. Pengujian Akses: Lakukan verifikasi dengan mengakses URL <https://doc.dki-uin.my.id> melalui browser menggunakan jaringan eksternal (di luar jaringan lokal VM) untuk memastikan web telah dapat diakses secara daring (online).

The screenshot shows a web interface for configuring a published application route. The left sidebar contains a navigation menu with options like Overview, Insights & Logs, Team & Resources, Networks, Overview, Tunnels & Mesh, Routes, Resolvers & Proxies, Access controls, Traffic policies, Email security, Data loss prevention, Browser isolation, Reusable components, Integrations, and Settings. The main content area is titled 'Add a published application route for home-lab' and includes a sub-header 'Publish a local application to the Internet via public hostname. DNS will be automatically configured. Learn more about routing to tunnels'. The configuration fields are as follows:

- Hostname:**
 - Subdomain (Optional):
 - Domain (Required):
 - Full hostname: doc.dki-uin.my.id
- Path (Optional):**
- How path matching works:** The path field uses regex patterns. Common examples:
 - Match all paths: leave empty
 - Match anywhere in path: `/blog` (matches `/blog/archive/blog/post`, etc.)
 - Match path prefix: `^/api/`
 - Match files by extension: `\.([a-z]{1,4})$`
- Service:**
 - Type (Required):
 - URL (Required):
 - The origin service to route traffic to (e.g., `https://localhost:8080`, `tcp://localhost:3306`)
- Origin request and connection settings** (expandable section)